

209. Minimum Size subarray sum:

Given an array of positive integers `nums` and a positive integer `target`, return the **minimal length** of a subarray whose sum is greater than or equal to `target`. If there is no such subarray, return `0` instead.

Example 1:

Input: `target = 7, nums = [2,3,1,2,4,3]`

Output: 2

Explanation: The subarray `[4,3]` has the minimal length under the problem constraint.

Example 2:

Input: `target = 4, nums = [1,4,4]`

Output: 1

Example 3:

Input: `target = 11, nums = [1,1,1,1,1,1,1,1]`

Output: 0

Constraints:

- `1 <= target <= 109`
- `1 <= nums.length <= 105`
- `1 <= nums[i] <= 104`

Solution: time = $O(n)$ and space = $O(1)$:

```
class Solution:
    def minSubArrayLen(self, target: int, nums: List[int]) ->
```

```

int:

    n = len(nums)
    left = 0
    ans = n+1
    win = 0
    for right in range(n):
        win += nums[right]
        while win >= target:
            ans = min(ans, right - left + 1)
            win -= nums[left]
            left += 1
    if ans == n+1:
        return 0
    else:
        return ans

```